

Activity 1.3b Reflection on Basic Concepts and Fundamental Algorithms

Kevin Alexander Ramirez Mendez A00844255 10/Sep/2026

Data Structures & Algorithms

For the main problem we have that the bitacora.txt file is not ordered and has its own log records. For the solution we have to order the records chronologically and save the ordered result, after this we must display the records in the desired date and time interval. This requires comparing timestamps, our Date class we implemented compares year, month, day hour and minutes and then the Log class has overloaded operators for comparing. This program's main.cpp file opens the file, parses everything into a Log object, asks the user for the timestamps they need and then stores the records in a vector, after this we call insertion sort to write every ordered record to the new file.

The project is built using Makefile for easier compilation, it compiles everything in the project and puts the result in the bin folder, the current design of the project works perfectly and performs only one filtering pass using insertion sort. If the same log file had to answer many queries I would replace insertion sort with another sorting algorithm like merge sort and combine it with binary search, this would improve the algorithm time when running many queries, in this case it is not necessary because it is only one query.

Another consideration we need to have is memory, the program stores all the records in a vector before sorting them so the memory grows linearly based on the number of logs, we also didn't use merge sort because it needs temporary memory during its merge operations and needs extra space. If the file was larger another sorting method could divide the data into smaller blocks and then we can sort those blocks, after sorting them they could be merged.

The program can be checked with different date ranges, the correctness of the program depends on the input being interpreted consistently. Each line is divided into month, day, time, IP, domain and message. The date is converted into a Date object before any comparisons are made. This is important because comparing the original text would give wrong results if the months or days have different lengths or when the text is not sorted in chronological order.

To end my reflection I have to say that this project showed me how algorithms work and implemented in real life use. Algorithms cannot be judged only if they do the task correctly, we also need to take into account the time that they take and if they produce the right order. The most important part of using algorithms is to use the best algorithm for the task you currently need, while knowing that some algorithms work better with more data and others work just fine with less data. If we take into account all of these factors we can choose the correct algorithm.